Data Mining 2

Topic 02 : Feature Engineering

Lecture 03 : Introduction to Feature Engineering

Dr Kieran Murphy

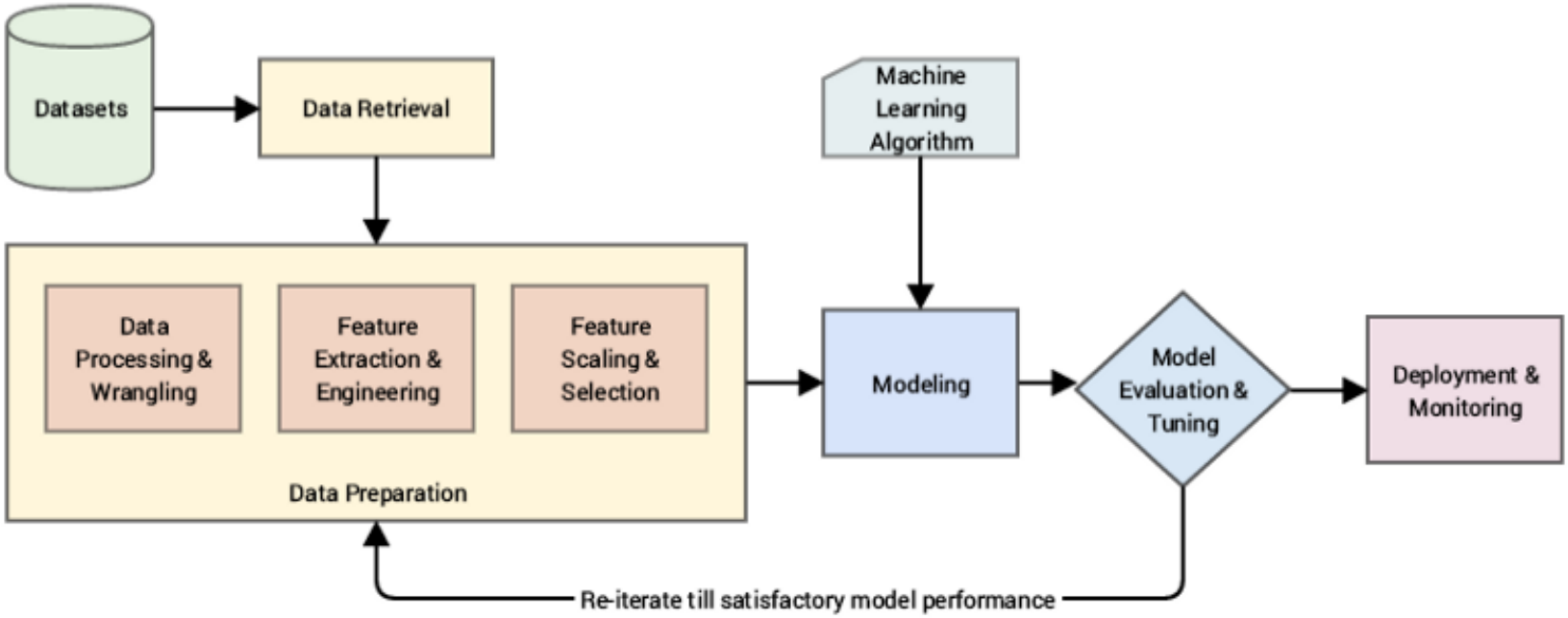
Department of Computing and Mathematics, Waterford Institute of Technology.
(Kieran.Murphy@setu.ie)

Spring Semester, 2024

Outline

- Need for feature engineering
- Generic feature engineering steps

Yet Another Data Mining Pipeline . . .



- Varying terminology.
  - Feature extraction and feature engineering are synonyms.
  - Feature transformation (or scaling) deals with scaling and transforming (to get symmetric (normal like) distributions.
  - Feature selection deal with picking subsets of given and generate features to use in the model.

$$\textbf{Feature Engineering} \approx \textbf{Extraction} \cup \textbf{Engineering} \cup \textbf{Transformations} \cup \textbf{Selection}$$

What is Feature Engineering?

Feature engineering

The process of transforming raw data into features that better represent the underlying problem to the predictive models, resulting in improved model accuracy on unseen data.

- is a representation problem*
  '*you have to turn your inputs into things the algorithm can understand*'

  – Shayne Miel

- is an Art
  '*Coming up with features is difficult, time-consuming, requires expert knowledge. "Applied machine learning" is basically feature engineering.*'

  – Andrew Ng

  '*. . . some machine learning projects succeed and some fail. What makes the difference? Easily the most important factor is the features used.*'

  – Pedro Domingos, (A Few Useful Things to Know about Machine Learning)"

*Recall the three components of a machine learning problem (Representation, Evaluation and Optimisation).

Why Feature Engineering?

- **Better representation of data**
  - Improved representation can be better understood by ML algorithms.
  - Better representations leads to improved visualisation — for example, compare the frequent word occurrences of a newspaper article as opposed to the raw text.
- **Better performing models**
  - The right features tend to give models that outperform other models no matter how complex the algorithm is.
  - In general if you have the right feature set, even a simple model will perform well and give desired results.

Better features make better models.

- **Essential for model building and evaluation**
  - Numerical data is slower on decision trees.
  - Categorical data not suitable for linear regression, recall one-hot encoding.
- **More flexibility on data types**
  - Want to build models on diverse data types (text, images, video).
- **Emphasis on the business and domain**
  - Better features are often motivated by domain experts (not data scientists), leading to more understandable (by humans) models.
  - Feature engineering emphasises to focus on the business and the domain of the problem when building features.

Example

At its simplest, feature engineering can be defined as the process of creating new features from the existing features in a dataset.

Consider a sample data that has details about a few items, such as their weight and price.

Item_ID	Item_Weight	Item_Price
FDA15	9.3	249.81
DRC01	5.9	48.27
FDN15	17.5	141.62
FDX07	19.2	182.10

```
Price_per_Weight=Item_Price/Item_Weight
```

Item_ID	Item_Weight	Item_Price	Price_per_Weight
FDA15	9.3	249.81	26.86
DRC01	5.9	48.27	8.15
FDN15	17.5	141.62	8.09
FDX07	19.2	182.10	9.48

If, for example, we were interested in a regression task, then, a similar effect could have been achieved by consider a suitable family of non-linear functions. However, generating features in advance allows us to stay with simpler and computationally less expensive families of functions.

Example: (Titanic dataset)

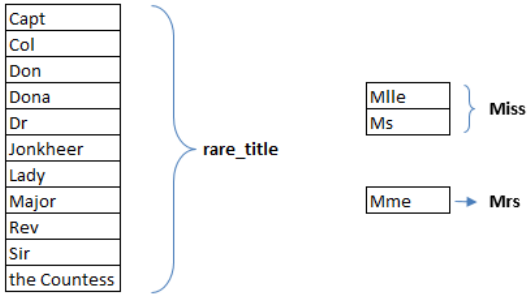
```
df = pd.read_csv('assets/train.csv')
df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.28		
				Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.925		
				Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
				Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

1 Have a (text) feature representing the passenger's name.

Title	Count
Capt	1
Col	4
Don	1
Dona	1
Dr	8
Jonkheer	1
Lady	1
Major	2
Master	61
Miss	260
Mlle	2
Mme	1
Mr	757
Mrs	197
Ms	2
Rev	8
Sir	1
the Countess	1

2 Rare categories can be merged under a single label – 'Dona', 'Lady', 'the Countess', 'Capt', 'Col', etc. .
Duplicate/redundant categories can be merged – the titles 'Mlle', and 'Ms' and can be merged under 'Miss'. 'Mme' can be merged with 'Mrs'.



3 New categorical feature with 5 unique values

Master	Miss	Mr	Mrs	rare_title
61	264	757	198	29

Importance of Feature Engineering

- Performance of a predictive model is heavily dependent on the quality of the features in the dataset used to train that model.
- Feature engineering can allow simpler models (linear vs non-linear) to provide required performance.

The Kaggle competition, Bike Sharing Demand Prediction, dealt with forecasting the rental demand based on historical usage patterns in relation with weather, time and other data. Smart feature engineering was instrumental[†] in securing a place in the top 5 percentile of the leaderboard, using:

Hour Bins Categorise the hour feature with number of bins determined using a decision tree.

Temp Bins: Similarly, a binned feature for the temperature variable for both registered and casual users.

Year Bins: 8 quarterly bins were created for a period of 2 years, i.e., quarterly bins.

Day Type: Days were categorised as 'weekday', 'weekend' or 'holiday'.

[†] Kaggle Bike Sharing Demand Prediction — How I got in top 5 percentile of participants?

Numeric Feature I

Numeric data typically represents data in the form of scalar values depicting

- observations, recordings or measurements
- frequencies or counts

It is very rich data but the pattern of interest may often be lost within the noise. Common feature engineering techniques are

- **Binarisation** — Generate a binary feature from a numeric feature.
- **Rounding** — reduce the precision in the data to simplify decision tree models, or as a step towards representing as categorical.
- **Binning** —- Divide numerical scale into bins (think histogram) to then convert to a categorical feature.
  - Bins can be fixed (equal) width or adaptive.
  - Bin width / number of bins can be determined using decision trees.

Example —- Binarisation

Consider building a recommendation system for song recommendations:

- Want to know if a person is interested or has listened to a particular song.
- If given a dataset with a feature `listen_count`, storing the number of time each individual has listened to each song, then we need to binarise our `listen_count` field as follows:

```
tmp = np.array(df["listen_count"])
tmp[tmp >= 1] = 1
df["listened"] = tmp
```

or (using more modern pandas)

```
df["listened"] = (df["listen_count"]>0).astype(int)
```

or (using sklearn)

```
from sklearn.preprocessing import Binarizer
bn = Binarizer(threshold=0.9)
tmp = bn.transform([df["listen_count"]])[0]
df["listened"] = tmp
```

Categorical Feature

Dealing with special categories

Imagine you have a categorical attribute, like `Item_Color` that can be 'Red', 'Blue' or 'Unknown'. All three categories are treated equally. But there are situations in which the 'unknown' should be treated separately — for example, when it arises from missing values in the original data and the instances of missing values is linked to the target.
In this case we need to create an additional binary feature to indicate the special category.

Merging Rare (infrequent) categories

See titles in Titanic dataset.

Encodings

- **Label Encoding** — label levels as $1, 2, \ldots$ based on order seen.
- **Ordinal Encoding** — label levels as $1, 2, \ldots$ based on natural order.
- **One-Hot Encoding** — separate binary indicator variable for each level.
- **Frequency Encoding** — level values are replaced by the frequency that the level appeared.
- **(K-Fold) Target Encoding** — level values are replaced by target statistic (mean) for that level.
- $\ldots$ many more $\ldots$

Frequency Encoding

In frequency encoding (count encoding), replace each level/category with the count of how often it appears in the dataset.

	Temperature	Color	Target	FreqEnc_Temperature
0	Hot	Red	1	0.4
1	Cold	Yellow	1	0.2
2	Very Hot	Blue	1	0.1
3	Warm	Blue	0	0.3
4	Hot	Red	1	0.4
5	Warm	Yellow	0	0.3
6	Warm	Red	1	0.3
7	Hot	Yellow	0	0.4
8	Hot	Yellow	1	0.4
9	Cold	Yellow	1	0.2

	Temperature	Color	Target	FreqEnc_Color
0	Hot	Red	1	0.3
1	Cold	Yellow	1	0.5
2	Very Hot	Blue	1	0.2
3	Warm	Blue	0	0.2
4	Hot	Red	1	0.3
5	Warm	Yellow	0	0.5
6	Warm	Red	1	0.3
7	Hot	Yellow	0	0.5
8	Hot	Yellow	1	0.5
9	Cold	Yellow	1	0.5

Frequency Encoding II

- ✔ **Effective with High Cardinality** — Since each category is replaced by its frequency, the encoding is not affected by number of categories, i.e., does not increase dimensionality (in contrast with One-Hot Encoding).
- ✔ **Preservation of Information** — The frequency of each category is preserved. This can be useful, e.g., in a sales dataset, the frequency of each product can be an indicator of the product's popularity.
- ✘ **Doesn't Handle Unseen Categories** — Levels/categories that do not appear in the train dataset are not encoded (worse than One-Hot encoding)
- ✘ **Loss of Unique Categories** — If two categories have the same frequency, they will be represented by the same value after frequency encoding.
- ✔ **Simplicity** — Calculation is trivial and not computationally expensive.
- ✔ **Works with all Models** — Suitable for both tree-based and non-tree-based models. (In contrast Ordinal Encoding does not work well with linear models).
- ✘ **Risk of Overfitting for Rare Categories** — Low frequency might not be a good representation of importance, leading to overfitting, where the model learns to rely too much on infrequent categories.
- ✘ **Not Suitable for Ordinal Variables** — Natural order is lost in frequency encoding.

Target Encoding

In target encoding replaces each level/category with the average value of the target variable for that category.

Rec-No	Temperature	Color	Target
1	Hot	Red	1
2	Cold	Yellow	1
3	Very Hot	Blue	1
4	Warm	Blue	0
5	Hot	Red	1
6	Warm	Yellow	0
7	Warm	Red	1
8	Hot	Yellow	0
9	Hot	Yellow	1
10	Cold	Yellow	1

Step-1	Sum of Target
Cold	2
Hot	3
Very Hot	1
Warm	1

Step-2	Count of Target
Cold	2
Hot	4
Very Hot	1
Warm	3

Step-3	Mean
Cold	1.00
Hot	0.75
Very Hot	1.00
Warm	0.33

Rec-No	Temperature	Color	Target
1	0.75	Red	1
2	1.00	Yellow	1
3	1.00	Blue	1
4	0.33	Blue	0
5	0.75	Red	1
6	0.33	Yellow	0
7	0.33	Red	1
8	0.75	Yellow	0
9	0.75	Yellow	1
10	1.00	Yellow	1

†https://towardsdatascience.com/all-about-categorical-variable-encoding-305f3361fd02

Target Encoding II

Target encoding has similar advantages and disadvantages as feature encoding. However target encoding has a fix for rare or unseen categories, based on smoothing.

$$\text{encoding} = \text{weight} \times \text{in_category} + (1 - \text{weight}) \times \text{overall}$$

where

- weight is a value between 0 and 1 calculated from the category frequency.
- in_category is the target mean value for rows with the given category value.
- in_category is the overall target mean value.

Then we compute weight using

$$\text{weight} = \frac{n}{n+m}$$

where

- n is the number of times that the category occurs in the data.
- m is the "smoothing factor".
  - Setting $m = 0$, means that weight $= 1$ so above formula reduces to

    $$\text{encoding} = \text{in_category}$$

    and no smoothing takes place.
  - Larger values of m put more weight on the overall estimate and less on the in_category estimate.

Date/Time Features

- Decomposing date into day, month, year, day of week
- Decomposing time in hour, minute, second.
- Subtracting dates to get age.
- Extracting day of week
- Categorising into workday/weekend — see Kaggle, Bike hire dataset
- Duration between two times — e.g., harmonic transformation

```
def make_harmonic_features(value, period=24):
    value *= 2 * np.pi / period
    return np.cos(value), np.sin(value)
```

This transformation preserves the distance between points, which is important for algorithms that estimate distance (kNN, SVM, k-means ...)

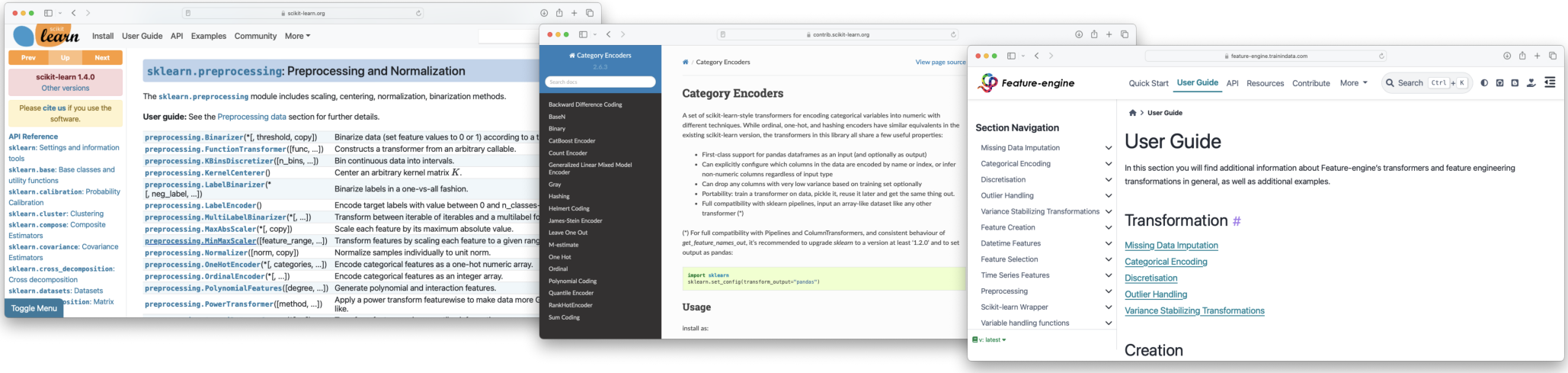
Libraries for Category Encoding

scikit-learn

- Most commom encoders and consistent style: `fit`, `transform`, and `fit_transform` methods.

Python module `category_encoders`

- extra scikit-learn-style transformers for encoding.

Python module `feature_engine` (via condo-forge)

- Only recently started using this, but it is comprehensive and compatable with scikit-learn.